Web Scraping using BeautifulSoup

1. Introduction to Web Scraping

Web scraping is the process of automatically extracting data from websites using programs or scripts.

Instead of manually copying information from webpages, a scraper collects the data automatically and stores it in a structured format such as:

- CSV
- Excel
- JSON
- Database

Real-Life Examples

Web scraping is used in:

- News aggregation websites
- Price comparison platforms
- Weather data collection
- Job portals
- Research and data analysis
- Social media monitoring
- E-commerce analytics

---

2. What is BeautifulSoup?

Beautiful Soup is a Python library used for:

- Parsing HTML and XML documents
- Navigating webpage structure
- Searching webpage elements
- Extracting required data easily

It converts messy webpage code into a structured tree format so developers can access tags, classes, IDs, links, tables, and text.

---

3. Why Use BeautifulSoup?

Advantages

- Easy to learn
- Beginner friendly
- Works with HTML and XML
- Excellent for small and medium scraping tasks
- Supports multiple parsers
- Helps clean poorly formatted HTML

Limitations

- Cannot interact with JavaScript-heavy websites directly
- Slower than some advanced scraping frameworks
- Requires additional libraries for dynamic content

---

4. Libraries Required

To perform web scraping using BeautifulSoup, we commonly use:

Library	Purpose
requests	Sends HTTP requests to websites
BeautifulSoup	Parses HTML content
pandas	Stores and analyzes extracted data

---

5. Installation

Install the required libraries using pip.

```
pip install beautifulsoup4
pip install requests
pip install pandas
```

---

6. Basic Workflow of Web Scraping

Step-by-Step Process

```
1. Send request to webpage
2. Receive HTML content
3. Parse HTML using BeautifulSoup
4. Locate required tags/elements
5. Extract data
6. Store data in file/database
```

---

7. Understanding HTML Structure

Before scraping, students must understand HTML basics.

Common HTML Tags

Tag	Purpose
<html>	Root document
<head>	Metadata
<title>	Webpage title
<body>	Main content
<h1> to <h6>	Headings
<p>	Paragraph
<a>	Hyperlink
<table>	Table
	Section/container
	Inline container

---

8. Creating the First BeautifulSoup Program

Example: Extract Website Title

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"

response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

print(soup.title.text)
```

---

9. Explanation of Code

requests.get(url)

Sends request to webpage.

response.text

Returns webpage HTML source code.

BeautifulSoup()

Parses HTML document.

html.parser

Python’s built-in HTML parser.

soup.title.text

Extracts title text.

---

10. Parsers in BeautifulSoup

BeautifulSoup supports different parsers.

Parser	Description
html.parser	Built-in parser
lxml	Faster parser
html5lib	Very tolerant parser

Example:

```
soup = BeautifulSoup(html, "lxml")
```

---

11. Finding Elements

BeautifulSoup provides various methods to locate elements.

A. find()

Returns first matching element.

```
soup.find("h1")
```

B. find_all()

Returns all matching elements.

```
soup.find_all("p")
```

C. Find by Class

```
soup.find("div", class_="content")
```

D. Find by ID

```
soup.find(id="main")
```

---

12. Extracting Text

```
paragraph = soup.find("p")

print(paragraph.text)
```

OR

```
print(paragraph.get_text())
```

---

13. Extracting Links

```
links = soup.find_all("a")

for link in links:
    print(link.get("href"))
```

---

14. Extracting Images

```
images = soup.find_all("img")

for image in images:
    print(image.get("src"))
```

---

15. Working with Tables

HTML Table Structure

```
<table>
   <tr>
      <td>Data</td>
   </tr>
</table>
```

Scraping Table Data

```
table = soup.find("table")

rows = table.find_all("tr")

for row in rows:
    cols = row.find_all("td")

    data = [col.text for col in cols]

    print(data)
```

---

16. Example: Scraping Quotes Website

```
import requests
from bs4 import BeautifulSoup

url = "http://quotes.toscrape.com"

response = requests.get(url)
```

```
soup = BeautifulSoup(response.text, "html.parser")

quotes = soup.find_all("span", class_="text")

for quote in quotes:
    print(quote.text)
```

---

17. Extracting Multiple Data Fields

```
import requests
from bs4 import BeautifulSoup

url = "http://quotes.toscrape.com"

response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

quote_blocks = soup.find_all("div", class_="quote")

for block in quote_blocks:

    quote = block.find("span", class_="text").text

    author = block.find("small", class_="author").text

    print("Quote:", quote)
    print("Author:", author)
```
